CHECK POINT

AI SECURITY STACK MAPPER

Admin Setup Guide

Installation, configuration & administration

v1.0.0

June 2026

1. Prerequisites

- **Node.js 20+** and npm
- **Docker + Docker Compose** — for the one-command full-stack / Postgres path
- *(Optional)* **Ollama** or any OpenAI-compatible local LLM runtime, for live model evaluation

2. Choose How To Run It

Option A — Docker Compose (recommended)

One command brings up **Postgres + API + nginx-served frontend**:

```
docker compose up --build      # → http://localhost:8080
```

The API auto-applies the schema and seeds demo data. Tear down with `docker compose down` (add `-v` to drop the database volume).

Option B — Full Stack, Local Dev (SQLite)

```
# 1) Backend
cd server
npm install
cp .env.example .env
npm run setup        # prisma generate + migrate deploy + seed
npm start            # API on http://localhost:4000

# 2) Frontend (separate terminal)
cd ..
npm install
npm run dev          # http://localhost:5173 (proxies /api → :4000)
```

Option C — Frontend Only (offline demo)

```
npm install && npm run dev     # http://localhost:5173
```

Runs entirely on seed data + in-browser mocks — the header shows **Local mode** and no login is required.

3. Environment Variables (`server/.env`)

VARIABLE	PURPOSE	DEFAULT
`DATABASE_URL`	Prisma connection string	`file:./dev.db`
`PORT`	API port	4000
`CORS_ORIGIN`	Allowed browser origin(s)	http://localhost:5173
`JWT_SECRET`	Session signing secret — **set in production**	ephemeral
`NODE_ENV`	`production` blocks demo seeding (§5)	unset
`ALLOW_SEED`	`true` to allow seeding in production	unset
`CHECKPOINT_TE_API_KEY` / `_BASE_URL`	Check Point Threat Emulation	mock
`CHECKPOINT_FILE_SCAN_API_KEY` / `_BASE_URL`	Check Point TE File Scan	mock
`LAKERA_GUARD_API_KEY` / `_BASE_URL`	Lakera Guard	mock

> Secrets are read at runtime and never hardcoded. In production, supply them via your platform's secret store and terminate TLS in front of the API so the `Secure` session cookies apply.

4. Database & Migrations

- Schema: `server/prisma/schema.prisma` ; committed migrations under `server/prisma/migrations/` .
- Apply: `npm run db:migrate` (`prisma migrate deploy`). Seed: `npm run db:seed` . `npm run setup` does generate + migrate + seed.
- **Postgres:** point `DATABASE_URL` at Postgres (Docker path does this); the schema is provider-portable.

5. First Admin & Demo Accounts

Demo accounts (dev/seed only) — all use password `Demo!1234` :

EMAIL	ROLE
owner@imbsys.com	Owner
appsec@imbsys.com	AppSec
soc@imbsys.com	SOC
viewer@imbsys.com	Viewer

Production: seeding is blocked when `NODE_ENV=production` (unless `ALLOW_SEED=true`). Create your first **Owner** with the bootstrap CLI:

```
cd server
npm run create:admin -- --email you@org.com --name "You" --password 'StrongPass!23'
```

The same command resets an existing user to Owner with a new password. Manage everyone else from **Settings** → **User Management**.

6. Roles & Permissions

ROLE	CAN DO
Owner	Everything, incl. integration toggles & user management
AppSec	Edit controls, risks, models; run file scans & prompt inspections
SOC	Edit risks/incidents; run scans & inspections; read-only elsewhere
Viewer	Read-only

Enforced server-side (`requirePerm` → 401/403) and mirrored in the UI.

7. Vendor Integrations (Mock → Live)

Integrations run in **mock mode** until keys are configured (§3). Once a key + base URL are set, the integration flips to **live** automatically and calls run server-side (keys never reach the browser). Status is shown on the **Integrations** page.

8. Local LLM Evaluation (Optional)

```
ollama pull llama3.2:1b        # any small model
```

In the app: **Model Coverage** → **Register model** with deployment **local**, endpoint `http://localhost:11434/api/generate`, model name `llama3.2:1b`, then click **Evaluate**. OpenAI-compatible runtimes (vLLM, LM Studio, LocalAI, TGI) work via their `/v1/chat/completions` or `/v1/completions` endpoint. Unreachable endpoints fall back to a simulated evaluation.

9. CI & The Security-Score Gate

CI runs type-check, build, unit + integration tests, and an **AI Security Score gate** that fails the build below a threshold:

```
cd server
SCORE_MIN=60 SCORE_PROJECT=proj-copilot npm run score:gate
```

10. Production Hardening Checklist

- ▪ Set a strong `JWT_SECRET` and `NODE_ENV=production`.
- ▪ Terminate **TLS** in front of the API (enables `Secure` cookies).
- ▪ Use managed **Postgres** with committed migrations + backups.
- ▪ Replace demo accounts; create the real Owner via `create:admin`.
- ▪ Supply vendor keys from a secret store; confirm integrations show **live**.
- ▪ Lock `CORS_ORIGIN` to your real frontend origin.
- ▪ Run `npm test` (frontend + server) and the score gate in CI.

11. Troubleshooting

SYMPTOM	FIX
Login won't accept demo creds	DB not seeded — run `npm run db:seed`
Logged out after API restart	`JWT_SECRET` not set (ephemeral secret rotated)
Header shows **Local mode**	Backend unreachable — start the API / check `CORS_ORIGIN`
Integration shows **mock**	No API key configured for that vendor (expected)
Evaluation says **simulated**	Model endpoint unreachable from the server
Docker API container exits	Check `docker compose logs api` (DB must be healthy first)